

The Monolingual Assumption: A Diagnostic Study of Bug Localization in Polyglot Software Systems

Abstract—Modern software systems are polyglot in nature, utilizing multiple programming languages for implementation, configuration, and testing. Despite this, most bug localization techniques are designed and evaluated under a “monolingual assumption”, overlooking the potential impact of task-level polyglotism, where a single bug fix spans multiple file types. This study presents the first comprehensive empirical investigation into the architectural feasibility of state-of-the-art (SOTA) bug localization models in polyglot software settings. We will evaluate both language-specific and language-agnostic models through controlled experiments on Java and Python projects from the BettleBox dataset. To enable fine-grained analysis, we introduce a bug-level categorization scheme that distinguishes between Mono-fix (single-language) and Poly-fix (multi-language) bug reports, and compare model performance across these categories. Additionally, we construct a Mono-fix-all dataset to simulate traditional “polyglot-blind” evaluation settings. By contrasting this with the original bug dataset, we quantify the hidden performance inflation in existing benchmarks. This study will reveal key semantic and architectural limitations in existing SOTA models, offering insights into their limited real-world applicability and highlighting new directions for future research.

Index Terms—bug localization, polyglot software, multi-language software, bugs, polyglot bug reports

I. INTRODUCTION

Polyglot software development, the practice of using multiple programming languages within a single project, is now commonplace in modern software engineering. This practice has been documented for decades; studies show that over 96% of GitHub open-source projects use two or more programming languages [1]. While this approach offers flexibility and can enhance developer productivity [2], it also introduces significant complexity. Studies indicate that polyglotism increases the likelihood of functional defects and security vulnerabilities [3], [4]. This complexity necessitates robust, automated tools that can support the polyglot nature of modern projects to ensure software quality. This need extends to critical maintenance activities, such as bug localization, raising the question of how existing techniques perform in polyglot settings.

Bug localization is the task of identifying buggy source files corresponding to a bug report, is fundamentally a retrieval and ranking problem. Automated approaches to this task are valuable, as they can significantly reduce the time developers spend locating bugs, allowing more time for resolution [5]. Despite the prevalence of polyglotism, most bug localization techniques are developed and evaluated under monolingual assumptions, where both the codebase and bug reports are treated as single-language artifacts. For example, bug reports from Java project, whose fixes refer only to Java source files.

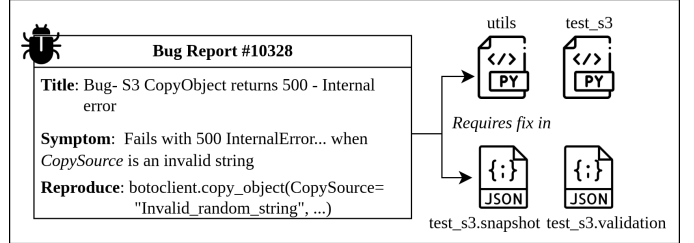


Fig. 1. Polyglot bug example from the project *localstack/localstack*.

At the same time, the adoption of automated bug localization tools in industry remains notably low [6]. We posit that this gap may stem from a disconnect between current techniques and the characteristics of polyglot systems, motivating our investigation into their robustness in polyglot settings.

Consider the bug report from the GitHub repository *localstack/localstack*¹ shown in Fig. 1. The issue describes a *500-Internal Error* triggered by the *S3 CopyObject* API when it receives a malformed input string, where a *400-Invalid Argument* error is expected. To correctly fix this bug, a developer must patch the error-handling logic in the Python source code (*utils.py*), add a corresponding regression test (*test_s3.py*), and update the expected output, which is stored in separate JSON files (*test_s3.snapshot.json* and *test_s3.validation.json*). This bug is therefore a Poly-fix task, as its true ground truth spans both the primary language (*.py*) and a secondary configuration language (*.json*). This example illustrates a complex, real-world fix that a traditional bug localization model may struggle to identify due to architectural and semantic constraints.

This study investigates how current SOTA bug localization techniques perform on polyglot projects, particularly when bug fixes span multiple programming languages. We hypothesize that the performance of SOTA bug localization models may be systematically overestimated due to a monolingual assumption whose validity in polyglot contexts remains untested. The disconnect between research assumptions and real-world practice presents a potential risk. In Regression Test Selection (RTS), recent work has shown that monolingual tools can be “unsafe” when applied to polyglot projects [4]. Bug localization literature often makes similar assumptions, evaluating models in monolingual settings. For example, mapping bug reports in Java projects exclusively to *.java* files. However, as illustrated in Fig. 1, real-world fixes frequently involve multiple file types, introducing a critical and underexplored challenge. This

¹<https://github.com/localstack/localstack/issues/10328>

may lead to two distinct, yet unmeasured, failure modes: (1) *Task-Level Failure*, where models are architecturally or semantically incapable of localizing *Poly-fix* bugs spanning source code, configuration files, and test-data schemas; and (2) *Project-Level Failure*, where polyglot “corpus noise” degrades performance in cross-project settings even for monolingual fixes. This study is the first to formally define and empirically evaluate these failure modes.

II. BACKGROUND AND RELATED WORK

A. Bug Localization

Bug localization (BL) techniques operate at various granularities (e.g., method, class, file); this study focuses on file-level localization, a widely adopted and practical setting in SOTA research. Among the diverse families of BL approaches, including program spectra and dynamic analysis techniques [7]; we focus on Information Retrieval-Based Bug Localization (IRBL), a dominant static paradigm that treats bug reports as natural language queries and source files as document [8]. IRBL models compute textual and semantic similarity between the report and codebase, to return a ranked list of likely buggy files [6]. However, this task is non-trivial due to the lexical gap between natural language and source code, and by the “cold-start” problem in projects lacking historical bug-fix data [6].

Recent advances in deep learning (DL) have led to significant improvements in IRBL performance. For example, HyLoc [9] was the first to apply deep neural networks to reduce lexical gap between bug reports and source code. BL-GAN [10], incorporates Abstract Syntax Trees (ASTs) and employs Graph Convolution Network to capture structural semantics. RLocator [11] frames it as a reinforcement learning problem, optimizing ranking metrics via a Markov decision process.

Despite these advancements, no prior work has examined the impact of *task-level polyglotism* in bug localization. Existing evaluations implicitly assume that bug fixes are monolingual, overlooking the complexity of real-world Poly-fix scenarios. This study is the first to formally evaluate SOTA BL models under polyglot conditions.

B. Polyglotism in Software Engineering

Software engineering literature uses varied terms such as “multilingual,” “cross-language,” and “hybrid-programming”; to describe the use of multiple languages within a single project [12]. In this study, we adopt the term *polyglotism* with formal definitions provided in Section IV-A.

Empirical studies confirm that polyglotism is the norm, not the exception. Tomassetti and Torchiano found that 96% of GitHub repositories in their sample used at least two programming languages [1]. Subsequent work identified “cross-language links” as a frequent source of bugs [13], and Stack Overflow analyses revealed that developers often struggle with complex, cross-language failures involving data conversion, interfacing, and memory access mechanisms [3]. While these studies highlight developer awareness of polyglot challenges, they do not evaluate their impact on automated software engineering tools. Recent research in software testing has

begun to address this gap. Houdaille et al. [14], examined the challenges of testing polyglot programs, and BabelRTS [4] demonstrated that applying RTS tools to polyglot projects can be “unsafe” as inter-language dependencies can be missed. Building on this precedent, future bug localization models should also move towards polyglot-aware techniques.

C. Retriever-Reranker Architecture

Real-world bug localization faces the “needle in a haystack” challenge: each bug report must be matched against thousands of source files in a project snapshot. Evaluating all (bug report, source file) pairs is computationally infeasible for DL-based SOTA models. To address this, we adopt a scalable two-stage retriever-reranker architecture. A lightweight retriever first selects a Top-K subset of candidate files, which are then reranked by a more complex SOTA model.

For retrieval, we construct a FAISS-based vector database storing embeddings for all unique file blobs across historical project snapshots. This enables time-specific querying for each bug report. Embeddings are generated using BAAI/bge-code-v1², a 1.5B LLM-based model, by chunking files, embedding each chunk, and averaging. The retriever returns the Top-K most semantically similar files, with K empirically determined via RQ0 (see Section III). Reranking is performed by five selected SOTA models. During training we ensure “perfect recall” by force-including any missing ground truth files in the candidate set. During testing, the reranker operates on the raw Top-K set without augmentation, simulating real-world deployment. Final performance depends on both retriever’s recall ($C@K$ from RQ0) and the reranker precision.

III. HYPOTHESES AND RESEARCH QUESTIONS

This study is designed to systematically deconstruct the impact of polyglotism. We formalize our investigation into the following RQs and testable hypotheses.

RQ0: (Preliminary) What is the recall ceiling of the two-stage retrieval architecture?

Rationale: The two-stage retriever-reranker design is a computational necessity. However, the SOTA reranker models (e.g., BLAZE) can only rank the candidates files provided by the retriever (FAISS). This RQ serves as a mandatory validation step to measure the “recall ceiling,” the maximum achievable performance of our system. Establishing this baseline is essential for contextualizing downstream results.

H₀: There exists a computationally feasible candidate size $K \leq 500$ that achieves a high recall ceiling ($C@K \geq 95\%$) for all bug report categories and for *Poly-fix* bug set, we hypothesize will be lower due to the semantic dissimilarity of non-code files.

RQ1: What are the architectural limitations of SOTA language-specific models in polyglot bug localization?

Rationale: A subset of SOTA models, such as COOBA relies on parsing source code into ASTs. This creates a testable hypothesis regarding their feasibility in a polyglot environment

²<https://huggingface.co/BAAI/bge-code-v1>

that includes non-parsable files (like *.yaml*, *.sql*, or *.gradle*).

H_{1.1} (Feasibility): Language-specific models may encounter execution failures on the *Overall* and *Poly-fix* bug sets due to parser limitations when handling non-primary language files. We hypothesize that such failures will highlight architectural constraints in applying these models to polyglot contexts.

H_{1.2} (Performance Bias): When evaluated only on the subsets they can process (*Mono-fix* and *Mono-fix-all*), language-specific models are expected to show high performance. We hypothesize that this performance may be biased, as the *Mono-fix-all* set excludes polyglot files.

RQ2: What is the impact of poly-fix bugs on the performance of language-agnostic models in a within-project (WP) setting?

Rationale: This RQ represents our primary controlled experiment. Unlike language-specific models, language-agnostic models can process all file types. By comparing on our defined bug report categories within the same project, we controls for project-specific variables (e.g., domain, coding style) and isolates the impact of the fix type.

H_{2.1} (Core Difficulty): Language-agnostic models will exhibit statistically significantly lower performance on the balanced *Poly-fix* set compared to the balanced *Mono-fix* set, indicating that polyglot fixes are inherently harder to localize.

H_{2.2} (The “Hidden Error”): For text-based models, performance on the *Mono-fix-all* dataset (a “traditional simulation”) will be statistically significantly higher than on the *Overall* dataset (reflecting “true reality”). This experiment aims to quantify the degree to which prior polyglot-blind evaluations may have overstated model effectiveness.

RQ3: To what extent do task-level and project-level polyglotism impact the generalization of language-agnostic models in a cross-project (CP) setting?

Rationale: This RQ examines whether the performance drop observed in RQ2 reflects a broader, generalizable limitation. It targets two distinct failure modes in CP contexts: (1) whether the task-level challenge of *Poly-fix* bugs persists or worsens, and (2) whether project-level polyglotism introduces “corpus noise” that degrades performance, even for *Mono-fix* bugs. We define corpus noise as the semantic ambiguity and expanded search space introduced by non-primary language files in the source and target project.

H_{3.1} (CP Generalization): The performance gap observed in *H_{2.1}* will persist and be statistically significantly larger across all 72 CP pairs, indicating that the challenge is widespread and amplified in transfer-learning scenarios.

H_{3.2} (Corpus Noise): Project-level polyglotism in the target project will negatively impact performance, even for *Mono-fix* bugs. We hypothesize that performance on *Mono-fix* bugs in Mono-Poly pairs will be statistically significantly lower than in Mono-Mono pairs.

RQ4: What mechanisms and factors contribute to performance degradation on Poly-fix bug reports?

H_{4.1} (File-Type): The failure set defined as bug reports for which all models fail will be disproportionately composed of fixes involving configuration files (*.json*, *.xml*), build scripts

(*.gradle*), and database/schema files (*.sql*). We hypothesize that these failures reflect semantic limitations, as models are not trained on the vocabulary or structure of these file types.

H_{4.2} (Structural Failure): A significant portion of failures will be attributable to structural constraints, such as the bug-relevant change (diff) occurring beyond the reranker’s input limit (e.g., 512 tokens). These limitations are typically invisible in traditional *Mono-fix* scenarios.

H_{4.3} (Semantic Failure) : The failure set will have bugs where the semantic link between the bug report and the non-primary file is non-trivial, making them harder to localize.

IV. EXPERIMENTAL METHODOLOGY

A. Definitions

In this section, we formally define the key terminologies to provide a consistent framework for designing our experimental settings and interpreting the results.

i) Primary Language : Let P be a software project and $L_{valid}(P)$ be the set of all valid source code, configuration, and build languages in P . Let $Bytes(l)$ be the total byte count for a given language $l \in L_{valid}(P)$, as reported by GitHub’s linguist library. The *primary language* is the single language l that maximizes this function:

$$L_{primary} = \arg \max_{l \in L_{valid}(P)} (Bytes(l)) \quad (1)$$

ii) Secondary Languages: Given the set of all valid languages $L_{valid}(P)$ and $L_{primary}$, the set of $L_{secondary}$ is defined as all other valid languages in the project:

$$L_{secondary} = \frac{L_{valid}(P)}{L_{primary}} \quad (2)$$

iii) Monolingual Project: A project P is classified as Monolingual if the byte-count contribution of its $L_{primary}$ exceeds a 95% threshold of the total bytes. Let $Bytes_{total} = \sum_{l \in L_{valid}(P)} Bytes(l)$

$$P \text{ is Monolingual if, } \frac{Bytes(L_{primary})}{Bytes_{total}} > 0.95 \quad (3)$$

iv) Polyglot Project: A project P is classified as *Polyglot* if the byte-count contribution of its $L_{primary}$ is less than or equal to the 95% threshold.

$$P \text{ is Polyglot if, } \frac{Bytes(L_{primary})}{Bytes_{total}} \leq 0.95 \quad (4)$$

v) Mono-fix Bug Reports: Let B be a bug report and $F(B)$ be the set of valid ground truth file extensions for its fix. B is classified as a *Mono-fix* bug report if its fix contains files of exactly one extension, and that extension is the $L_{primary}$.

$$B \text{ is Mono-fix if, } |F(B)| = 1 \wedge F(B) = L_{primary} \quad (5)$$

vi) Poly-fix Bug Reports: A bug report B is classified as a *Poly-fix* bug report if its fix contains two or more valid file extensions, at least one of which is the $L_{primary}$.

$$B \text{ is Poly-fix if, } |F(B)| \geq 2 \wedge L_{primary} \in F(B) \quad (6)$$

vii) Other-fix Bug Reports: A bug report B is classified as a *Other-fix* bug report if its set of valid ground truth file extensions $F(B)$ is not empty, but contains no files from the Primary Language ($L_{primary}$).

$$B \text{ is Other-fix if } L_{primary} \notin F(B) \wedge |F(B)| \geq 1 \quad (7)$$

viii) Overall Bug Reports (Dataset): The dataset $S_{overall}$ for a project P is the complete, unaltered set of all bug reports B from the filtered dataset, where B is classified as either *Mono-fix* or *Poly-fix*.

$$S_{overall} = \{B \mid B \text{ is Mono-fix or } B \text{ is Poly-fix}\} \quad (8)$$

ix) Mono-fix-all Bug Reports (Dataset): The *Mono-fix-all* dataset is a simulated version of the *Overall* dataset, designed to replicate traditional, polyglot-blind bug localization evaluations. For every bug report B_i in the *Overall* dataset, its ground truth $F(B_i)$ is altered to $F'(B_i)$ by stripping all non-primary language files.

$$F'(B_i) = F(B_i) \cap L_{primary} \quad (9)$$

x) Language-Agnostic Models: Models which treat source code as a sequence of raw text tokens, just as they would a natural language bug report. This architecture allows them to process any file type (*.java*, *.yaml*, *.sql*, *.gradle*) without a parser, making them “language-agnostic” by design.

xi) Language-Specific Models: Models whose core architecture requires parsing a source file into a representation such as ASTs to extract structural features (like function calls or file-level graphs). This reliance on a parser ties them to a single, specific language, making them “language-specific.”

B. Variables

The **independent variables (IVs)** are categorical. The first is *Model Architecture*, with two categories: *Language-Agnostic* and *Language-Specific*. The second IV is *Bug Report Type*, central to our WP and CP analyses, with four levels: *Overall*, *Mono-fix-all*, *Mono-fix*, and *Poly-fix*, as defined in Section IV-A. The third IV, used exclusively in the CP experiment, is *CP Pair Category*, which classifies each source-target pair into one of four levels: *Mono-Mono*, *Poly-Poly*, *Mono-Poly*, and *Poly-Mono*. The **dependent variables (DVs)** are standard performance metrics for IRBL: (1) *Mean Reciprocal Rank (MRR)*, which captures the average rank of the first correctly retrieved ground truth file; (2) *Mean Average Precision (MAP)*, which reflects the overall ranking quality and is sensitive to multiple ground truth files; and (3) *Top@K* (for $K = 1, 5, 10$), which measures the proportion of bug reports with at least one correct file ranked within the Top-K results.

We identified several **confounding variables**, for instance, the choice of ‘Reranker Model Architecture’ we aggregate findings across five SOTA models, enabling paradigm-level conclusions rather than model-specific ones. To mitigate ‘Data Scarcity’ in projects like *ansible/ansible* (Table I), we design two WP experiments: a ‘WP-Balanced’ (projects with $N > 30$ in both *Mono-fix* and *Poly-fix* sets), and ‘WP-Unbalanced’ (all 13 projects). This dual setup also addresses ‘Data Imbalance’

by applying balanced subsampling for the *Mono-fix* vs. *Poly-fix* comparison, and the full, unbalanced data for the *Overall* vs. *Mono-fix-all* comparison. Finally, project-specific confounding factors (e.g., domain, size, team) are inherently controlled through a within-subject WP design, comparing bug types originating from the same project.

C. Dataset Selection

We select the BeetleBox dataset [15] for this study due to its unique suitability. It offers a diverse set of 29 projects spanning five programming languages (C++, Go, Java, JavaScript, and Python); and is the only known bug localization dataset with ground truth files across multiple languages and file types. These file types include source code ($L_{primary}$), scripting, build, configuration & data, web & UI, server-side template, schema & definition, database-related, and test/spec files. This granularity enables analysis beyond the project-level, allowing us to examine task-level polyglotism reflective of real-world bug-fix scenarios often overlooked in prior datasets.

To construct a focused and statistically robust experimental set, we apply a multi-stage filtration process. First, we restrict the dataset to projects whose ($L_{primary}$) is either Java or Python, representing the statically-typed and dynamically-typed ecosystems in bug localization literature. This yields 17 projects (8 Java, 9 Python). Next, we retain only projects with at least 100 bug reports to ensure sufficient data for WP analysis and balanced subsampling. This results in a final set of 13 projects (6 Java, 7 Python), used for all WP and CP experiments. Detailed statistics and categorizations are provided in Table I. Each project is then classified as *Monolingual* or *Polyglot* (see Section IV-A), based on its codebase language distribution reported by GitHub’s Linguist library³. We adopt this byte-count-based metric over custom Lines of Code (LOC) calculations, as it offers reproducibility and language-agnostic precision, avoiding biases from language verbosity.

D. Experimental Plan

To comprehensively evaluate the impact of polyglotism, we will conduct our experiments in two distinct settings: Within-Project (WP) and Cross-Project (CP). The WP setting, where a model is trained and tested on data from the same project, serves as our primary controlled experiment. While widely used, WP is not always scalable and may suffer from a cold-start problem in new projects. To address this, we also implement a CP experiment simulating a realistic transfer-learning scenario: a model trained on a *source* project is fine-tuned on a small portion of a *target* project and evaluated on the remaining data [8], [16]. We have selected five SOTA bug localization models for this study: TRANP_CNN [8], COOBA [16], BLAZE [15], FLIM [17], and BL-GAN [10]. A brief architectural overview of these models is provided in Table II. Utilizing these SOTA models, this study experimental protocol will follow a five-step process designed to address our research questions (RQs) and test the associated hypotheses.

³<https://github.com/github-linguist/linguist>

TABLE I
OVERVIEW OF SELECTED BETTLEBOX PROJECTS AND THEIR CORRESPONDING STATISTICS FORMING THE BASIS OF OUR DATASET.

Project	Primary Language	Bug Reports					Project Category
		Overall	Mono-fix-all	Mono-fix	Poly-fix	Other-fix	
elastic/elasticsearch	Java	970	722	272	450	248	Monolingual
apache/dolphinscheduler	Java	1363	698	501	197	665	Polyglot
apache/dubbo	Java	379	253	226	27	126	Monolingual
liquibase/liquibase	Java	241	199	99	100	42	Polyglot
OpenRefine/OpenRefine	Java	620	190	130	60	430	Polyglot
SeleniumHQ/selenium	Java	134	36	38	8	98	Polyglot
apache/airflow	Python	1181	917	603	314	264	Polyglot
ccxt/ccxt	Python	690	40	12	28	650	Polyglot
localstack/localstack	Python	472	433	262	171	39	Monolingual
PostHog/posthog	Python	403	101	56	45	302	Polyglot
ansible/ansible	Python	768	627	150	477	141	Polyglot
langchain-ai/langchain	Python	130	112	80	32	18	Monolingual
odoo/odoo	Python	115	80	63	17	35	Polyglot

TABLE II
SELECTED BUG LOCALIZATION MODELS WITH ARCHITECTURE SUMMARIES AND CATEGORY LABELS.

Model	Architecture Summary	Category
BLAZE	A Transformer-based (LLM) model that treats source code and bug reports as text sequences, using dynamic chunking to handle long files.	Language Agnostic
BL-GAN	A semi-supervised Generative Adversarial Network. It explicitly parse each source file into an AST and uses the AST's structure to create tokens.	Language Specific
COOBA	An adversarial transfer learning model. It parses code into an AST, builds a graph, and uses a Graph Convolutional Network (GCN) to create code representations.	Language Specific
FLIM	A graph-based model. It parse each source file into an AST to extract all function-level information and build a file-level graph.	Language Agnostic
TRANP_CNN	A deep transfer learning model using two separate Convolutional Neural Networks (CNNs). It treats code as a flat text corpus.	Language Agnostic

Step 1- Retriever Validation: To empirically determine the optimal number of candidate files per bug report, we validate our retriever (FAISS + 1.5BLLM) as a standalone component. We construct a FAISS index from unique file blobs across snapshots of all 13 projects. Using the test set of bug reports, we retrieve the Top-K most similar files and measure $C@K$ for $K = [50, 100, 200, 300, 500]$. The primary K will be chosen as the smallest K such that $C@K \geq 0.95$ for *Mono-fix-all* and ≥ 0.85 *Poly-fix* bug reports, to balance recall and computational feasibility. A plot of K vs. $C@K$ will justify our final choice of K for all subsequent experiments. If no $K \leq 500$ satisfies both thresholds, it proves that retrieval recall is imperfect, and reranker results will be interpreted accordingly. We anticipate selecting a small K , as larger values increase training time, introduce data imbalance, and reduce reranker precision in Top-10 rankings. All analyses will also report sensitivity across K values to ensure conclusions are not specific to one cutoff.

Step 2- Bug Report Categorization: We begin with ground-truth filtering and pre-process all ground truth files to retain only code-related extensions (e.g., .java, .py, .sql, .gradle) and excluding non-code artifacts such as documentation

(.md), images (.png), or version-control artifacts (.gitignore). Each project's bug reports are categorized into four sets: *Overall*, *Mono-fix-all*, *Mono-fix*, and *Poly-fix*. To ensure fair comparison for *Mono-fix* vs. *Poly-fix* ($H_{2.1}$ and $H_{3.1}$), we will construct balanced datasets. For each project, we identify the smaller set size (N_{min}) and randomly sample N_{min} bug reports from the larger set to create a 1:1 balanced test set. This procedure is repeated across all cross-validation folds.

Step 3- Within-Project Evaluation: We will run five SOTA reranker models on each of the 13 projects using 5-fold cross-validation (or an 80/20 train/test split for categories with $N < 50$ bugs). During training, the Top-K candidate files are retrieved for each bug, and any missing ground truth files are added to ensure exposure to positive examples. We will also include a control experiment of (no augmentation) for at least one representative project to test robustness. During testing, the reranker ranks the Top-K candidates without augmentation, simulating real-world conditions. The output is a ranked list for each bug report in the test set. The language-agnostic models will run on all bug report sets, while language-specific models will run only on the *Mono-fix* and *Mono-fix-all* sets, per $H_{1.1}$.

Step 4- Cross-Project Evaluation: We generate 72 source-target pairs (30 Java, 42 Python), each labeled by project type (e.g., Mono-Poly). For each pair and model, we train on 100% of the source project's data plus 20% of the target's training data, adding missing ground truth files as in Step 3. The fine-tuned model is then tested on the remaining 80% of the target's test data, producing ranked lists for each bug report.

Step 5- Qualitative Analysis: Following WP evaluation, we aggregate model performance to identify two diagnostic sets: *Failure Set* and *Success Set* with 50 *Poly-fix* bug reports each with lowest/highest average performance metrics. This yields a 100-bug dataset for qualitative analysis, prioritizing reports from the best and worst-performing Java and Python projects.

E. Analysis Plan

This section outlines how experimental data will be used to test each hypothesis. We will use MAP, MRR, and Top@K ($K=1, 5, 10$) as DVs for all quantitative analyses. Statistical significance will be assessed using non-parametric tests (e.g.,

Mann-Whitney U and Wilcoxon signed rank tests) with an $\alpha = 0.05$, and effect sizes will be reported (e.g., Cliff’s Delta) alongside p-values.

RQ1 - Architectural Limitations: To test $H_{1.1}$, we will run language-specific models on the *Overall* and *Poly-fix* test sets from the WP experiment, measuring execution feasibility. For $H_{1.2}$, we compare *Mono-fix-all* performance from language-specific models and *Overall* performance from the language-agnostic models. Using Mann-Whitney U, we test whether $\text{Perf}_{AST}(\text{Mono-fix-all}) > \text{Perf}_{Text}(\text{Overall})$, arguing that language-specific models are evaluated on relatively simple bug report compare to language-agnostic models which handles polyglot data.

RQ2 - WP Performance: To test $H_{2.1}$, we use balanced WP test sets to compare performance distributions of language-agnostic models, testing whether if, $\text{Perf}(\text{Mono-fix}) > \text{Perf}(\text{Poly-fix})$. For $H_{2.2}$, we use unbalanced WP sets to test whether, $\text{Perf}(\text{Mono-fix-all}) > \text{Perf}(\text{Overall})$, highlighting the overestimation risk in polyglot-blind evaluations.

RQ3 - CP Generalization: To test $H_{3.1}$, we will aggregate results from all 72 CP pairs and compare balanced *Mono-fix* vs. *Poly-fix* performance, also reporting whether this gap exceeds that observed in WP. For $H_{3.2}$, we isolate *Mono-fix* results and compare distributions across Mono-Mono and Mono-Poly CP pairs, testing whether $\text{Perf}(\text{Mono-fix}, \text{Mono-Mono}) > \text{Perf}(\text{Mono-fix}, \text{Mono-Poly})$, thereby quantifying the impact of corpus noise.

RQ4 - Qualitative Diagnosis: To test $H_{4.1}$, we manually categorize bug types in the 100-bug sample (e.g., Source + Config, Source + Build) and use a Chi-squared test to assess whether the *failure set* contains a significantly higher proportion of Config/Build/Schema fixes. For $H_{4.2}$, we perform a traceability check on each *failure set* bug to determine whether line-level changes in non-primary files fall outside the model’s token limit (e.g., 512 tokens), reporting the truncation-correlated failure rate. For $H_{4.3}$ we will perform a content analysis of the bug report text for the *failure set*, determining the semantic link between them and $L_{secondary}$ file is indirect and non-trivial making the model harder to localization.

V. THREATS TO VALIDITY

We have identified the following potential threats to the study’s validity. We will mitigate each threat through specific, pre-defined protocols in our experimental and analysis plan.

A. Construct Validity

This category addresses the alignment between our definitions and the real-world phenomena they aim to represent. The primary constructs used in this study, bug report categories and project-level classifications, are novel but grounded in preliminary data analysis and tailored to our hypotheses. These definitions are not arbitrary; they are essential for operationalizing our research questions. To ensure reproducibility, all categorization scripts are publicly available. The Mono-fix-all category simulates “traditional” polyglot-blind research, it is not intended as a direct replication of prior studies. Similarly,

our project-level classification uses a 95% byte-count threshold as a heuristic to simplify categorization. This threshold is not a definitive architectural label, as shown by *elastic/elasticsearch* is *Mono-lingual* (99.5% Java, yet 49.4% Poly-fix bugs). The categorization of SOTA model into two categories due to their architecturally distinct paradigms naturally allow us to Test H_1 and make a paradigm-level claim.

B. Internal Validity

Internal validity concerns the strength of causal inferences drawn from our experimental design. A primary threat is disentangling retriever failure from reranker failure. We address this through RQ0, which measures the retriever’s recall ceiling ($C@K$), allowing all reranker performance to be interpreted relative to this “maximum theoretical performance.” Another threat arises from data imbalance between *Mono-fix* and *Poly-fix* sets in certain projects *ansible/ansible* (150 vs. 477 bugs). We address this through a dual analysis strategy: balanced subsampling for hypothesis-driven comparisons ($H_{2.1}$, $H_{3.1}$), and unbalanced data for simulating traditional evaluation settings ($H_{2.2}$). To reduce subjectivity in RQ4’s qualitative analysis, two researchers will independently annotate 50 “hard” and 50 “easy” bugs. We will report inter-rater reliability (e.g., Cohen’s Kappa) and resolve disagreements through consensus.

C. External Validity

This category of threats concerns the generalizability of our findings. The primary threat is our reliance on a single dataset, BeetleBox. We will acknowledge this limitation but justify the choice, as to our knowledge, BeetleBox is the only large-scale, public dataset that provides the *Poly-fix* ground truth necessary to conduct this study. A second threat is our limited scope of 13 projects from two languages (Java and Python). We will explicitly state that our findings may not generalize to other ecosystems (e.g., C++). However, by selecting two popular languages from different paradigms (statically-typed vs. dynamically-typed), we provide a strong initial foundation for generalizability and will position our study as a “first step” that calls for future work. Finally, our CP fine-tuning strategy (100% Source + 20% Target) is a deliberate methodological choice. Based on literature, this approach is preferred over zero-shot, which often yields uninformative results. This strategy gives the models a “realistic chance” to succeed, which will make their (hypothesized) failure on *Poly-fix* bugs a more significant and telling finding.

REFERENCES

- [1] F. Tomassetti and M. Torchiano, “An empirical assessment of polyglotism in github,” in *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, 2014, pp. 1–4.
- [2] M. Abidi, M. Grichi, and F. Khomh, “Behind the scenes: developers’ perception of multi-language practices,” in *Proceedings of the 29th annual international conference on computer science and software engineering*, 2019, pp. 72–81.
- [3] H. Yang, Y. Nong, S. Wang, and H. Cai, “Multi-language software development: Issues, challenges, and solutions,” *IEEE Transactions on Software Engineering*, vol. 50, no. 3, pp. 512–533, 2024.
- [4] G. Maurina, W. Cazzola, and S. Ghosh, “Babelrts: Polyglot regression test selection,” *IEEE Transactions on Software Engineering*, 2025.

- [5] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, 2016.
- [6] F. Niu, C. Li, K. Liu, X. Xia, and D. Lo, "When deep learning meets information retrieval-based bug localization: A survey," *ACM Computing Surveys*, vol. 57, no. 11, pp. 1–41, 2025.
- [7] D. Zou, J. Liang, Y. Xiong, M. D. Ernst, and L. Zhang, "An empirical study of fault localization families and their combinations," *IEEE Transactions on Software Engineering*, vol. 47, no. 2, pp. 332–347, 2019.
- [8] X. Huo, F. Thung, M. Li, D. Lo, and S.-T. Shi, "Deep transfer bug localization," *IEEE Transactions on software engineering*, vol. 47, no. 7, pp. 1368–1380, 2019.
- [9] A. N. Lam, A. T. Nguyen, H. A. Nguyen, and T. N. Nguyen, "Combining deep learning with information retrieval to localize buggy files for bug reports (n)," in *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2015, pp. 476–481.
- [10] Z. Zhu, H. Tong, Y. Wang, and Y. Li, "Bl-gan: Semi-supervised bug localization via generative adversarial network," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 11, pp. 11 112–11 125, 2022.
- [11] P. Chakraborty, M. Alfadel, and M. Nagappan, "Rlocator: Reinforcement learning for bug localization," *IEEE Transactions on Software Engineering*, 2024.
- [12] G. Mussbacher, B. Combemale, J. Kienzle, L. Burgueño, A. Garcia-Dominguez, J.-M. Jézéquel, G. Jouneaux, D.-E. Khelladi, S. Mosser, C. Pulgar *et al.*, "Polyglot software development: Wait, what?" *Ieee software*, vol. 41, no. 4, pp. 124–133, 2024.
- [13] P. Mayer, M. Kirsch, and M. A. Le, "On multi-language software development, cross-language links and accompanying tools: a survey of professional software developers," *Journal of Software Engineering Research and Development*, vol. 5, no. 1, p. 1, 2017.
- [14] P. Houdaille, D. E. Khelladi, B. Combemale, and G. Mussbacher, "On polyglot program testing," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 507–511.
- [15] P. Chakraborty, M. Alfadel, and M. Nagappan, "Blaze: Cross-language and cross-project bug localization via dynamic chunking and hard example learning," *IEEE Transactions on Software Engineering*, 2025.
- [16] Z. Zhu, Y. Li, H. Tong, and Y. Wang, "Cooba: Cross-project bug localization via adversarial transfer learning," in *IJCAI*, 2020.
- [17] H. Liang, D. Hang, and X. Li, "Modeling function-level interactions for file-level bug localization," *Empirical Software Engineering*, vol. 27, no. 7, p. 186, 2022.